Numerical Analysis Midterm Project

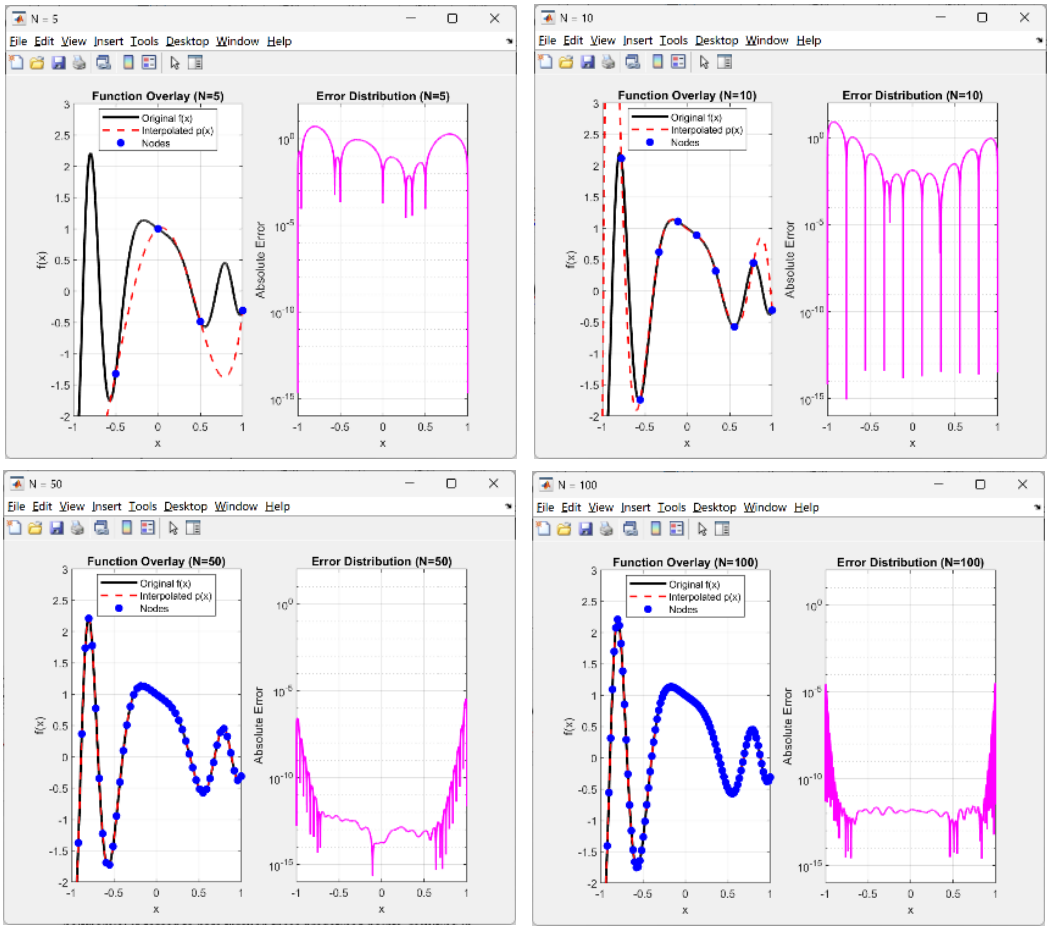
B12502027 CHENG-HSUAN LEE

(The contents of this report were all translated with the assistance of AI, and were deleted and modified by me.

The original Chinese version will be attached in the appendix at the end)

1.

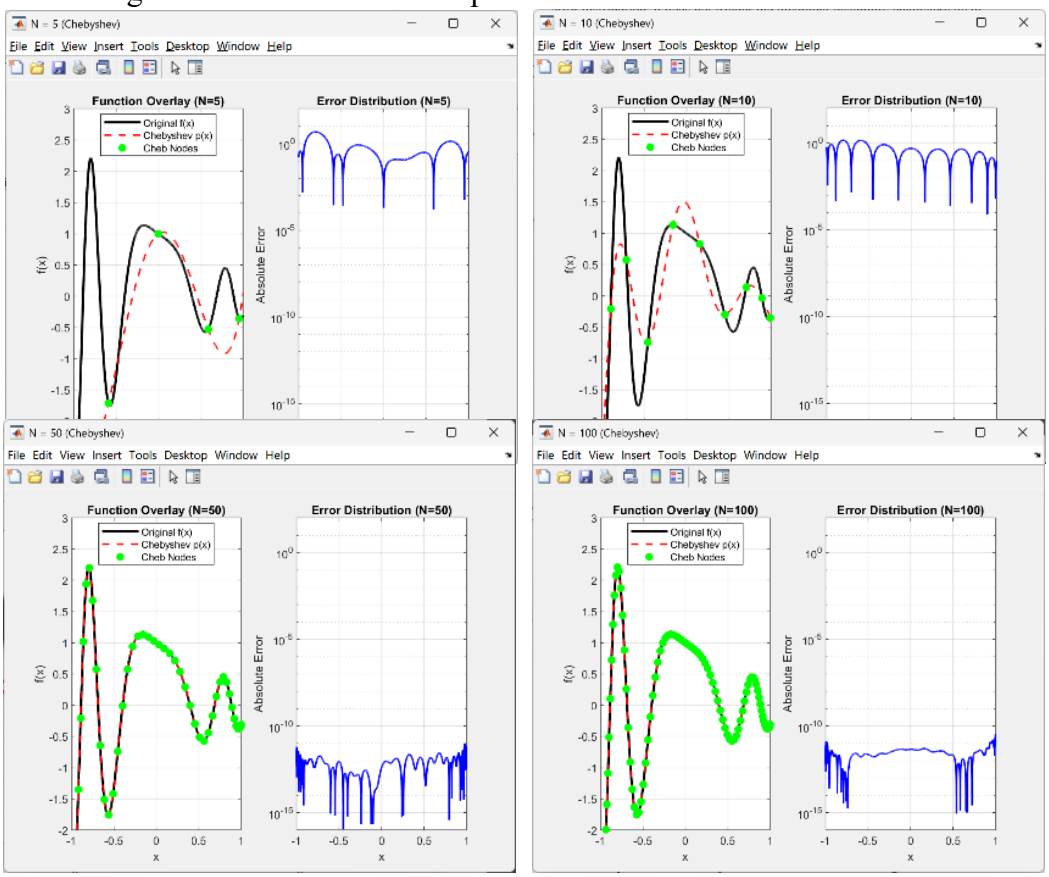
(a) .Following are the error distribution plots:



The numerical experiments conducted for interpolation point counts of $N = 5, 10, 50, 100$ provide a detailed perspective on how the distribution of points affects accuracy. In all four cases, it is evident that the absolute error approaches zero precisely at the interpolation nodes, as the polynomial is strictly constrained to pass through these coordinates. These instances manifest as sharp dips in the error distribution plots; however, due to the finite density of the computational grid, the error values at these nodes do not vanish entirely but remain at the level of machine precision. Excluding these nodal points, a distinct trend of "low error in the center, high error at the boundaries"

becomes increasingly prominent as N increases. This indicates that while increasing the number of interpolation points effectively stabilizes the error in the central region ($x \approx 0$), the requirement to force-fit evenly spaced nodes near the edges ($x \approx \pm 1$) causes obvious oscillations between those nodes, which leads to less reliable predictions in the boundary regions. In summary, simply increasing the number of interpolation points does not guarantee global uniform convergence, even if the goal is to enhance overall precision. Furthermore, upon a deeper investigation into Runge's Phenomenon, I speculate that the characteristics observed in these specific plots do not strictly constitute the phenomenon itself. Runge's phenomenon typically refers to a scenario where increasing the number of equidistant points causes errors at the boundaries to skyrocket to magnitudes such as 10^2 or 10^3. Observation of the $N = 100$ plot reveals that although the boundary errors are higher than those in the center—mirroring the distribution pattern associated with Runge's findings—the maximum error remains below a certain threshold and does not exhibit an uncontrolled exponential surge. Therefore, for this specific equation, while the evenly spaced sampling method results in a non-uniform error distribution, it does not trigger the divergent instability characteristic of Runge's phenomenon.

(b) . Following are the error distribution plots:



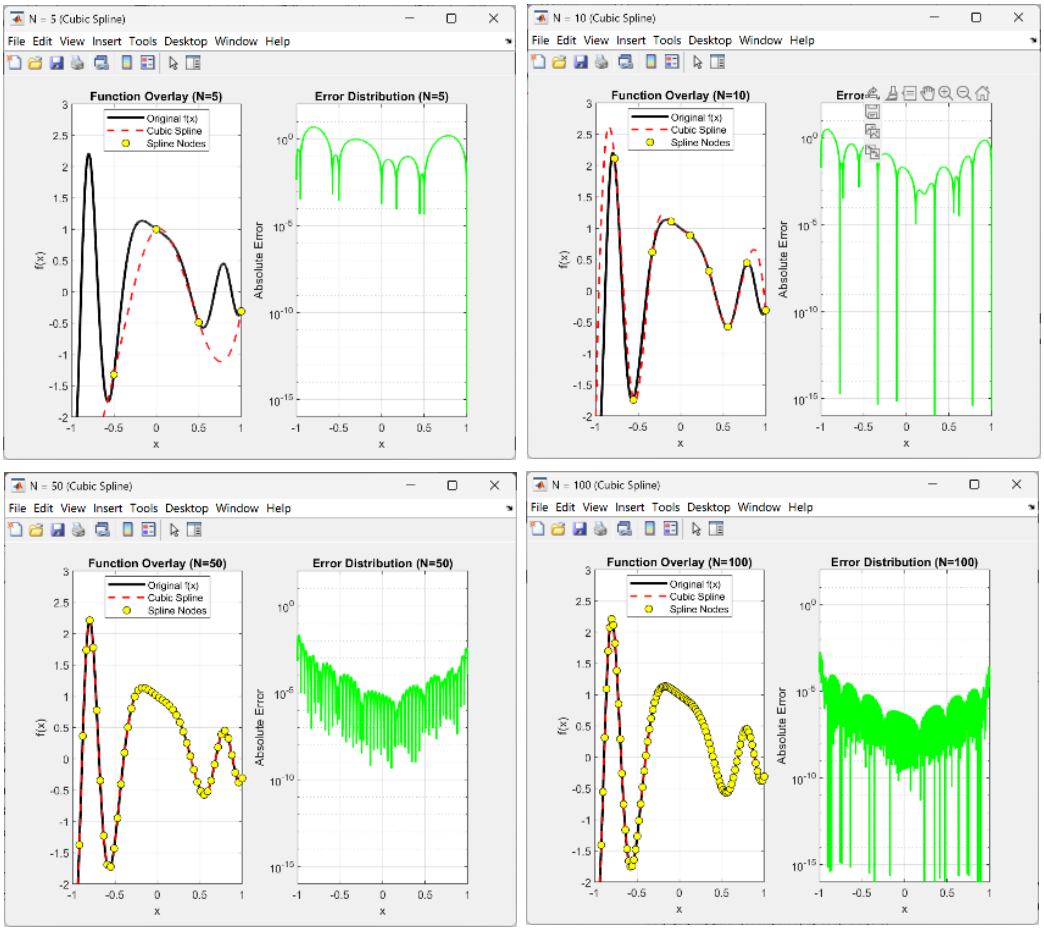
Similar to part (a), the error distributions were analyzed for $N = 5, 10, 50, 100$. In contrast to the uniformly distributed nodes used previously, Chebyshev nodes are defined as the projections of equidistant points from a semicircle onto a straight line. This geometric property creates a distribution that is sparse in the center and dense at the boundaries.

Based on the error distribution plots, it is evident that this specific node arrangement effectively suppresses oscillations by increasing the sampling density in the edge regions. This significantly mitigates the issue of error growth at the boundaries. Taking $N = 100$ as an example: while the error near the boundaries approaches 1 when using evenly spaced nodes, the error is controlled within the magnitude of 10^{-11} when utilizing Chebyshev nodes.

In summary, the use of Chebyshev nodes leads to a state of global convergence as N increases; the error diminishes uniformly across both the central and boundary regions, effectively suppressing the issue of non-uniform error distribution. This results in a much more stable and accurate approximation, proving that the choice of node distribution is critical to the stability of high-degree polynomial interpolation.

Furthermore, if the original function is susceptible to Runge's Phenomenon at the boundaries, this sampling method can avoid it quite effectively. The dense distribution of nodes near the edges significantly alleviates the error spikes characteristic of Runge's Phenomenon, yielding much more robust results than equidistant sampling.

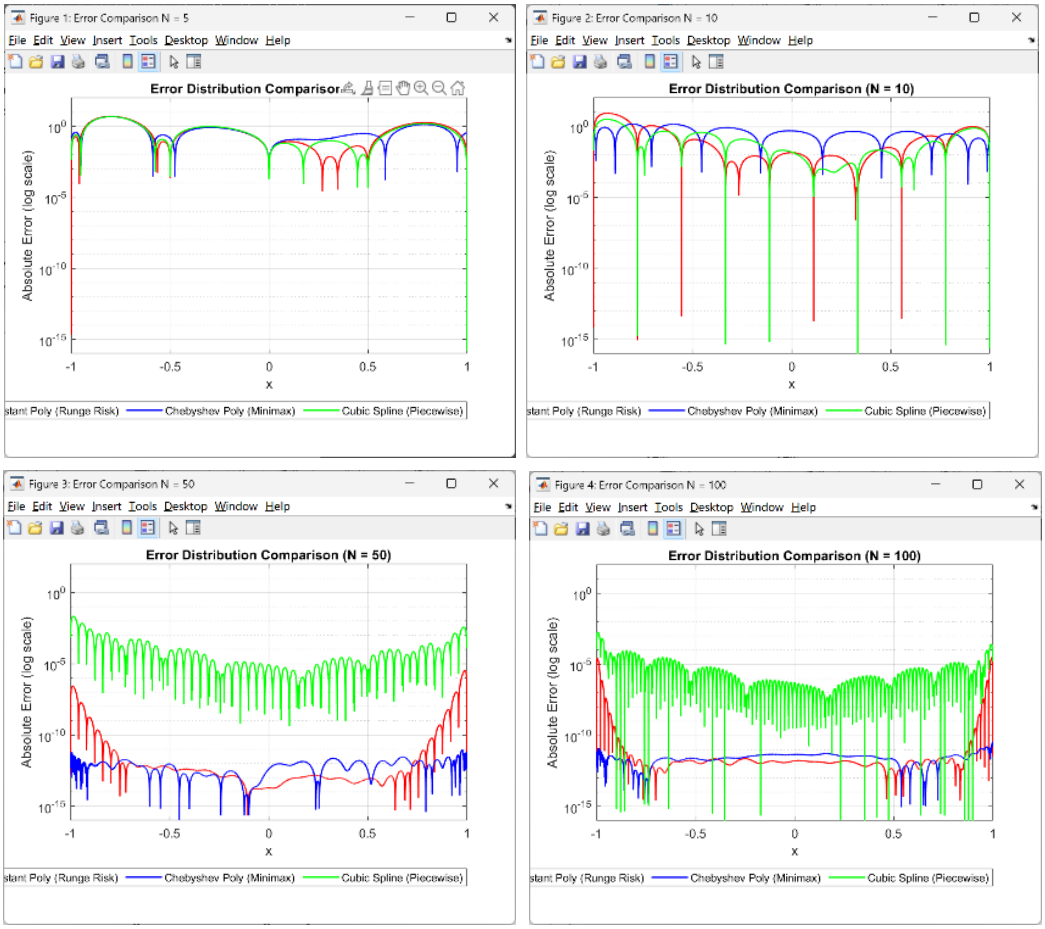
(c) . Following are the error distribution plots:



In contrast to the high-degree polynomial interpolation in part (a), which utilizes a single global curve, Cubic Spline Interpolation constructs a cubic polynomial between each pair of adjacent nodes. This approach ensures the continuity of both the first and second derivatives at the nodal points.

As observed from the four error distribution plots, although a slight "low in the center, high at the edges" trend persists as N increases, Cubic Spline Interpolation effectively mitigates the issue of excessive errors in the boundary regions compared to the results in part (a). This method successfully constrains the global error distribution within a controlled range. Furthermore, as N increases, the global error exhibits a uniform decline, demonstrating that when employing equidistant sampling, piecewise low-degree polynomials offer significantly higher stability than a single high-degree polynomial. Similar to the Chebyshev node method, piecewise low-degree polynomials are also effective at reducing the massive oscillations and errors associated with Runge's Phenomenon near the edges.

Error Discussion Across Three Methods:



Finally, an overlay of the error distributions for all three methods is presented for cross-comparison. It is evident that at low values of N, the errors across the three methods are comparable, with magnitudes hovering around 10^{1}. However, as N increases beyond 50, the two single high-degree polynomial methods yield significantly lower errors in the central region compared to the piecewise low-degree method, with a difference approaching 10^{-9} orders of magnitude.

Nevertheless, regarding the uniformity of error distribution, both Chebyshev nodes and Cubic Spline Interpolation significantly outperform equidistant nodes. As N approaches 100, the boundary errors of the equidistant method even become comparable to the overall error level of the Cubic Spline Interpolation. In conclusion, while single high-degree polynomials offer superior precision in the central segment as N increases, Chebyshev nodes and Cubic Spline Interpolation provide much better global error consistency once N is large enough to keep all errors within a controllable range.

2.

(a) . $g(x;a) = e^{-ax^2}$ and $f(x;a) = \sum_{m=-\infty}^{\infty} g(x - 2\pi m; a)$

For the continuous Fourier transform of $g(x;a)$, $G(s;a)$:

$$G(s;a) = \int_{-\infty}^{\infty} g(x;a)e^{-isx}dx = \int_{-\infty}^{\infty} e^{-ax^2}e^{-isx}dx$$

$$-ax^2 - isx = -a\left(x^2 + \frac{is}{a}x\right) = -a\left(x + \frac{is}{2a}\right)^2 - \frac{s^2}{4a}$$

Substitute into the integral $\rightarrow G(s;a) = e^{-\frac{s^2}{4a}} \int_{-\infty}^{\infty} e^{-a(x+\frac{is}{2a})^2}dx$

Knowing that $\int_{-\infty}^{\infty} e^{-au^2}du = \sqrt{\frac{\pi}{a}} \rightarrow G(s;a) = \sqrt{\frac{\pi}{a}}e^{-\frac{s^2}{4a}}$

Analytical expression for the Fourier series coefficients of $f(x;a)$, $\hat{f}_j(a)$:

$f(x;a) = \sum_{j=-\infty}^{\infty} \hat{f}_j(a)e^{ijx}$

$$\hat{f}_j(a) = \frac{1}{2\pi}\int_{-\pi}^{\pi} f(x;a)e^{-ijx}dx = \frac{1}{2\pi}\sum_{m=-\infty}^{\infty}\int_{-\pi}^{\pi} g(x - 2\pi m; a)e^{-ijx}dx$$

Let $u = x - 2\pi m, du = dx$

$$\rightarrow \hat{f}_j(a) = \frac{1}{2\pi}\sum_{m=-\infty}^{\infty}\int_{-\pi-2\pi m}^{\pi-2\pi m} g(u;a)e^{-iju}e^{-ij2\pi m}du$$

$$= \frac{1}{2\pi}\sum_{m=-\infty}^{\infty}\int_{-\pi-2\pi m}^{\pi-2\pi m} g(u;a)e^{-iju}du \quad (j, m \in \mathbb{Z}, e^{-ij2\pi m} = 1)$$

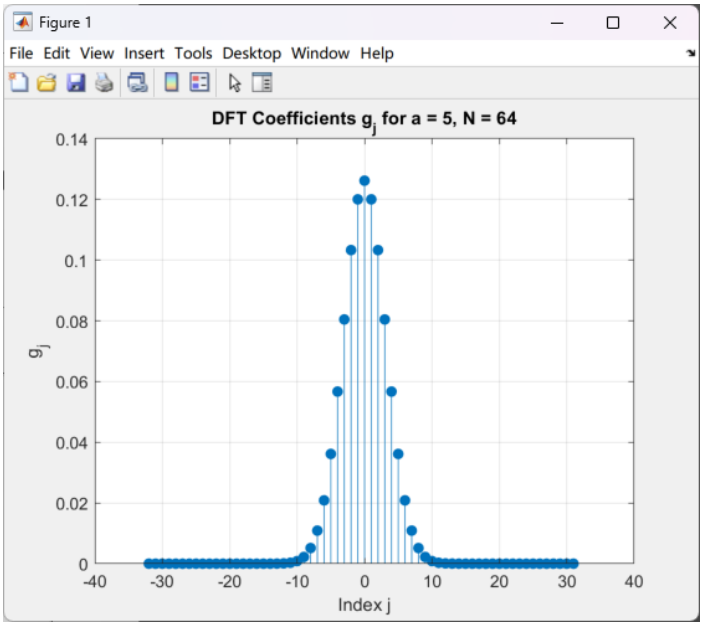
$$= \frac{1}{2\pi}\int_{-\infty}^{\infty} g(u;a)e^{-iju}du$$

Observing the two integral $\begin{cases} G(s;a) = \int_{-\infty}^{\infty} g(x;a)e^{-isx}dx \\ \hat{f}_j(a) = \frac{1}{2\pi}\int_{-\infty}^{\infty} g(u;a)e^{-iju}du \end{cases}$ it's easy to find out that $\hat{f}_j(a)$ are the value of $G(s;a)$ at the frequency point $s = j$, scaled by the factor of 2π

$$\rightarrow \hat{f}_j(a) = \frac{1}{2\pi}G(j;a) = \frac{1}{2\pi}\sqrt{\frac{\pi}{a}}e^{-\frac{j^2}{4a}}$$

(b) . Following are the DFT coefficients of $g(x;a)$:

Take a = 5, N = 64 for example

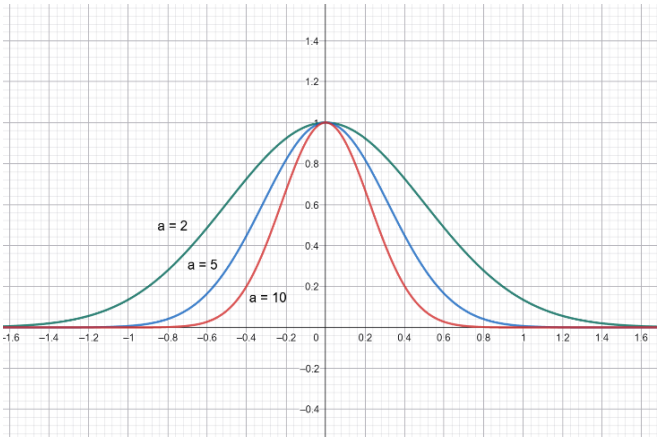


Verification:

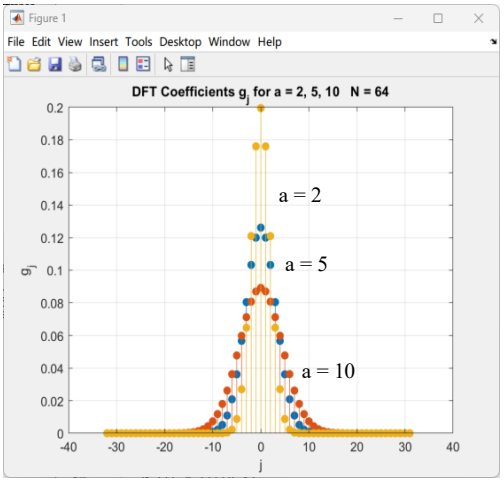
Based on the analytical expression derived in part (a), by substituting the parameters a = 5 and N = 64 into the formula, we can calculate the coefficient value at j = 0 equals to $\hat{f}_j(5) = \frac{1}{2\pi}\sqrt{\frac{\pi}{5}}e^{-\frac{0^2}{4\times 5}} \approx 0.126$. The resulting value is found to be closely consistent with the peak height observed at j = 0 in the output plot.

The following section presents the output and comparison of the DFT coefficient plots for a = 2, 5, and 10 while keeping N = 64 as constant:

$g(x;a)$ plots



DFT coefficients $\hat{g}_j(a;N)$



It can be observed that as the parameter a increases, the time-domain function $g(x;a)$ becomes increasingly localized and sharper. Consequently, the

distribution of the DFT coefficients $\hat{g}_j(a;N)$ (representing the frequency domain) becomes broader and flatter. This phenomenon is a direct manifestation of the scaling property of Fourier Transforms

(c) . To isolate and analyze the three types of potential errors—truncation of the Fourier coefficients, sampling/aliasing, and non-periodicity/leakage—we must compare the following three values:

$\frac{1}{2\pi}|G(s;a)|$: The continuous Fourier Transform (the ideal theoretical value without any numerical errors).
$\left|\hat{f}_j(a)\right|$: The Fourier series coefficients.
$\left|\hat{g}_j(a;N)\right|$: The DFT coefficients obtained via numerical computation.

Truncation Error:

Cause: The true continuous transform $G(s;a)$ extends infinitely across the frequency domain. However, when calculating the Fourier series, we only retain a finite range of frequencies $j \in (-\frac{N}{2}, \frac{N}{2} - 1)$. The error generated by discarding the high-frequency tail is the Truncation Error, which causes the difference between the ideal value $\frac{1}{2\pi}|G(s;a)|$ and the series coefficients $\frac{1}{2\pi}|G(s;a)|$.

Isolation of Error: By fixing N at a sufficiently large value and gradually increasing a from a small value, we can observe the effect. When a is very small, the function decays slowly in the frequency domain, leading to a significant difference between $\left|\hat{f}_j(a)\right|$ and $\frac{1}{2\pi}|G(s;a)|$ if high-frequency regions are neglected. As a increases, the decay rate in the high-frequency region increases, and the difference between $\left|\hat{f}_j(a)\right|$ and $\frac{1}{2\pi}|G(s;a)|$ diminishes noticeably. This demonstrates that the parameter a controls the Truncation Error.

Sampling/Aliasing Error:

Cause: The calculation of $\left|\hat{f}_j(a)\right|$ still assumes a continuous (integral) form, whereas the calculation of $\left|\hat{g}_j(a;N)\right|$ only samples N discrete points within a fixed range. According to the Nyquist Theorem, if the signal

bandwidth exceeds the sampling range, high-frequency components will "fold back" and disguise themselves as lower frequencies, causing interference between different frequencies. The existence of this error can be found by comparing $\left|\hat{f}_j(a)\right|$ and $\left|\hat{g}_j(a;N)\right|$ at lower frequencies.

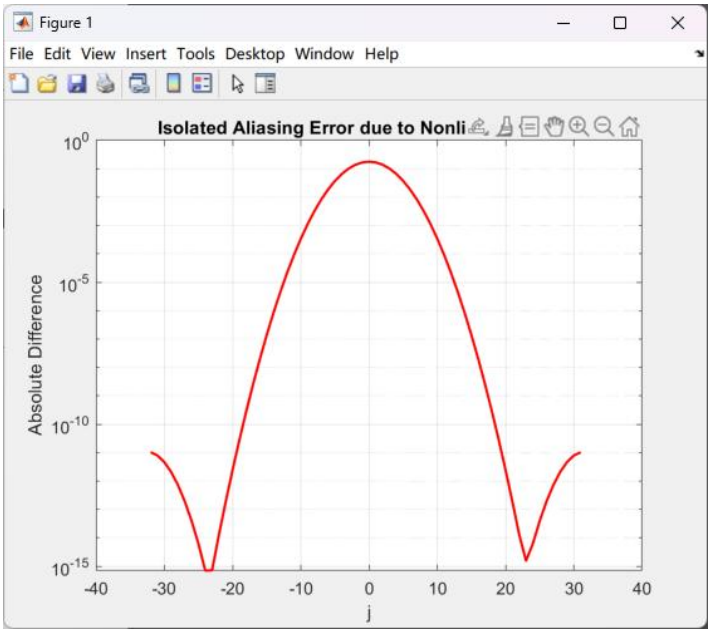
Isolation of Error: By fixing a large a (e.g., a = 10) and gradually increasing N, we can observe the trend. When N is small, the observation range is narrow. Since a=10 results in a wider spectrum, high-frequency components fold back and superimpose onto the low frequencies, causing $\left|\hat{g}_j(a;N)\right|$ at low frequencies to be significantly higher than the theoretical $\left|\hat{f}_j(a)\right|$. As N increases, the portion of the spectrum falling outside the range diminishes, and the aliasing error drops rapidly.

Non-periodicity/Leakage Error:

Cause: While the continuous function $G(s;a)$ is smooth and unbounded, the computation of the DFT coefficients $\left|\hat{g}_j(a;N)\right|$ is affected by the boundaries. If a discontinuity occurs when the signal is stitched together at the boundaries during periodic summation, a gap is created. According to Fourier analysis, any sharp discontinuity requires an infinite summation of high-frequency waves. This discontinuity subsequently disperses energy—which should originally be concentrated at specific frequencies—causing it to leak across the entire spectrum.

Isolation of Error: By fixing the value of N and gradually increasing a, we can isolate this effect. When a is small, the Gaussian function is relatively flat, potentially resulting in non-zero values at the boundaries that cause discontinuities. In this case, an overall upward shift in the error distribution can be observed. However, by increasing a, the Gaussian pulse becomes sharper, ensuring that it decays toward zero before reaching the boundaries. This effectively reduces the leakage error. Additionally, a window function can be applied to the signal to forcibly bring the values at both endpoints to zero. This ensures the periodicity of the signal within the observed domain, thereby effectively suppressing spectral leakage.

(d) . Following are the error due to the nonlinear product:



In this section, we first perform a direct FFT to obtain the DFT coefficients $\left|\tilde{h}_{j,1}(a;N)\right|$. We then recompute the coefficients $\left|\tilde{h}_{j,2}(a;N)\right|$ after applying the 3/2 De-aliasing Rule. By comparing the difference between these two, we can isolate the nonlinear aliasing error.

Direct Calculation ($\left|\tilde{h}_{j,1}(a;N)\right|$):

Assuming the maximum frequency component of $g(x;a)$ is k_{max}, the maximum frequency of its square $h(x;a) = g^2(x;a)$ becomes $k'_{max} = 2k_{max}$. If the original N sampling points are maintained, according to the Nyquist Theorem discussed in part (c), these high-frequency components exceeding the range will fold back into the low-frequency region, causing significant aliasing errors.

Calculation via the 3/2 De-aliasing Rule ($\left|\tilde{h}_{j,2}(a;N)\right|$):

The primary logic is to first perform a length N FFT. After transformation, the N spectral coefficients are placed into a larger array of length M = 3/2N, with zeros padded in the high-frequency regions (zero-padding). Performing an IFFT on these length M yields a finer sampling grid in the time domain. By conducting calculations on this refined grid, the extra high-frequency components have enough range to avoid folding back. Subsequently, a length M FFT is performed, followed by truncating the extra portions to retain only the coefficients corresponding to the original length N. Through this method, high-frequency components will not alias into the low-

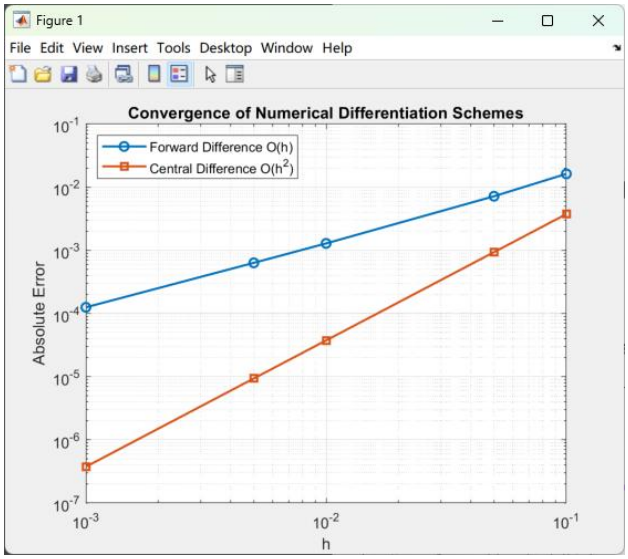
frequency region due to insufficient range, thus preventing aliasing error in the low-frequency coefficients.

Analysis of the Error Plot:

The plot illustrates the impact of the aliasing effect on the coefficients. It can be observed that the error is largest in the central low-frequency region. This can be viewed as a result of high frequencies folding back toward the center, causing the maximum impact on low frequencies. At approximately $j = \pm 23$, the error drops sharply; it is conjectured that this is near the Nyquist boundary, where certain high-frequency components fold back in a manner that results in the minimum magnitude of impact in this specific region.

3.

(a) . Following are the error of Forward Difference and Central Difference:



h	Forward Error	Central Error
0.1000	1.6258962253e-02	3.7485421130e-03
0.0500	7.1887110530e-03	9.3740886114e-04
0.0100	1.2875102709e-03	3.7499854159e-05
0.0050	6.3437629296e-04	9.3749908876e-06
0.0010	1.2537501046e-04	3.7500000172e-07

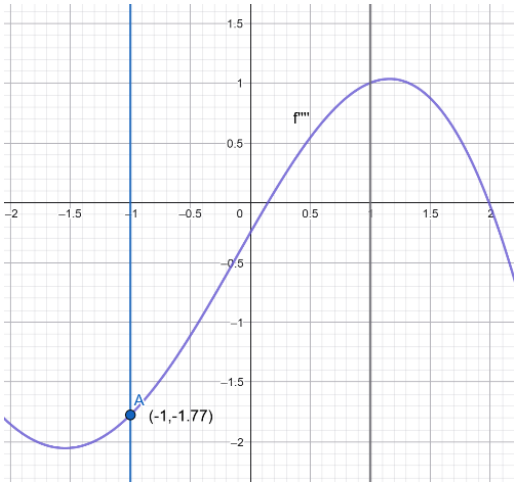
This section provides an analysis of the forward difference and central difference schemes. From a mathematical perspective, the error term for the forward difference, $R^2 f'(x_j)$, is directly proportional to the step size h (i.e., first-order accuracy, O(h)). This means that when h is reduced to 1/10, the error of the forward difference also decreases by approximately 1/10. In contrast, due to its symmetry, the central difference has an error term $R^3 f'(x_j)$ proportional to the square of the step size h (i.e., second-order accuracy, O(h^2)). Thus, when h is reduced to 1/10, the error of the central difference decreases by 1/100, theoretically providing much more accurate results.
The numerical output from the program fits perfectly with these theoretical derivations. By conducting an error analysis at x = 0, it is evident that as h gradually decreases, the error of the forward difference maintains a roughly linear relationship with h. Meanwhile, the error of the central difference exhibits a rapid, quadratic decay. As h becomes smaller, the difference between the two schemes becomes more pronounced, and the calculations provided by the central difference become significantly more accurate.

(b) . Following are the error of Gaussian quadrature method and 1/3 Simpson's rules:

```
Exact Integral: -2.5876005968
Gauss 2-point:  -2.5856740220 (Error: 1.93e-03)
Simpson 1/3:    -2.5905134391 (Error: 2.91e-03)
```

For this problem, since N=1, we utilize a 2-point Gaussian quadrature scheme. The sampling points correspond to the two zeros of the 2nd-order Legendre polynomial, $x = \pm\frac{1}{\sqrt{3}}$。. Theoretically, a Gaussian quadrature with N+1 points can achieve an algebraic precision of degree 2N+1. Therefore, this method can provide exact analytical results for polynomials of degree 3 or lower. Based on the numerical output from our program, the error using the 2-point Gauss integration is only 1.93×10^{-3}. However, according to the professor's lecture notes, if this method is used to integrate polynomials higher than the third degree, the error can reach 14.8% or more, resulting in a sudden drop in accuracy. In such cases, 2-point Gauss integration may no longer be the optimal choice.

On the other hand, Simpson's 1/3 rule is a formula based on evenly spaced sampling points. It performs a quadratic interpolation using three equidistant points (x = -1, 0, 1). Based on the derived formula, the global error is approximately: $Error \approx -\frac{(x_N - x_1)}{180}\overline{f^{(4)}}h^4$. This indicates that the error of Simpson's 1/3 rule is proportional to the fourth power of the step size h. Thus, if h is reduced by 1/2, the theoretical error should decrease to 1/16 of its original value. Additionally, we observe that the error is directly proportional to the fourth derivative $f^{(4)}$. If the function fluctuates significantly within the integration domain—meaning the magnitude of the fourth derivative is large—the error of this method will increase accordingly.



The figure above illustrates the fourth derivative $f^{(4)}$ of $f(x)$. It can be seen that within the integration range, the maximum value of $f^{(4)}$ is only 1.77. This magnitude is not large enough to cause a substantial impact on the numerical error. Our program's output confirms this, as the error for Simpson's 1/3 rule is merely2.91×10^{-3}. Although this error is slightly higher than that of the Gaussian quadrature, it remains well within a reasonable and acceptable range for numerical simulation.

(c) . Following are the output of Bisection method and Newton-Raphson method：

```
--- Searching for Root 1 ---
Bisection: Root = -3.6688767026, Iterations = 33
Newton:    Root = -3.6688767027, Iterations = 3

--- Searching for Root 2 ---
Bisection: Root = -5.7591312526, Iterations = 33
Newton:    Root = -5.7591312526, Iterations = 4
```

In this section, I utilize the Bisection Method (Linear Convergence, α = 1) and the Newton-Raphson Method (Quadratic Convergence, α = 2) to perform numerical root approximation.

We initially define a permissible error tolerance (10^{-10}) and determine the starting intervals or initial positions for both methods. This process is crucial as it involves narrowing down the root's range beforehand to prevent converging to an incorrect root. Since both methods can approach the true value through near-infinite iterations, our comparison method involves setting a fixed error tolerance and observing how many iterations each method requires to enter this range. Fewer iterations indicate h igher efficiency and better accuracy under a fixed number of iterations.

Based on the output results, it is observed that since the Bisection method follows linear convergence ($\alpha = 1$) , it requires more than thirty iterations to reach a precision of 10^{-10}, demonstrating relatively low efficiency. In contrast, the Newton-Raphson method exhibits quadratic convergence ($\alpha = 2$) , where the error of the subsequent stage is proportional to the square of the current error. Consequently, it requires fewer than five iterations to enter the tolerance range.

Subsequently, the program was modified to a fixed number of iterations (iter = 15) to observe the difference in error between the two methods under identical computational effort.

Method	Estimated Root	Absolute Error
Bisection (O(h))	-3.668861389160156	1.53135e-05
Newton (O(h^2))	-3.668876702710145	4.44089e-16

From the program output, it can be found that when both methods operate for 15 iterations simultaneously, the error of the Newton method (4.44×10^{-16}) is significantly smaller than that of the Bisection method (1.53×10^{-5}). This demonstrates that quadratic convergence is far superior to linear convergence in terms of computational efficiency.

Appendices

(The following AI prompts will be further modified for the code output by the AI. It is not a code that the AI can meet the requirements once after entering the following prompt)

B12502027_Q1_a (assisted by AI)

AI prompt:

Write a MATLAB program to analyze the polynomial interpolation error of f(x) = cos(10x^2) e^(-x) in the interval -1 ~ 1, and use polyfit to fit N = 5, 10, 50, 100 equidistant nodes. Each N should present two graphs, one is the superposition of the original function and the new fitting curve, and the other is the absolute error distribution.

Program:

```
% Q3 Interpolation Analysis: Fitting Overlay and Error Distribution
clear; clc;

f = @(x) cos(10*x.^2) .* exp(-x);
N_list = [5, 10, 50, 100];
x_grid = linspace(-1, 1, 10000);
y_true = f(x_grid);

for i = 1:length(N_list)
    nodes = N_list(i);
```

```
% 1. Generate equidistant interpolation points and calculate interpolation polynomials
    x_nodes = linspace(-1, 1, nodes);
    y_nodes = f(x_nodes);
    p = polyfit(x_nodes, y_nodes, nodes-1);
    y_interp = polyval(p, x_grid);
    error = abs(y_true - y_interp);

% Create a new window or sub-image (overlay is drawn on the left and error is drawn
on the right)
    figure('Name', ['N = ', num2str(nodes)], 'NumberTitle', 'off');

% --- Left picture: original function vs fitted function ---
    subplot(1, 2, 1);
    plot(x_grid, y_true, 'k-', 'LineWidth', 2, 'DisplayName', 'Original f(x)');
    hold on;
    plot(x_grid, y_interp, 'r--', 'LineWidth', 1.5, 'DisplayName', 'Interpolated p(x)');
    plot(x_nodes, y_nodes, 'bo', 'MarkerFaceColor', 'b', 'DisplayName', 'Nodes');
    grid on;
    xlabel('x'); ylabel('f(x)');
    title(['Function Overlay (N=', num2str(nodes), ')']);
    legend('Location', 'best');
ylim([-2, 3]); % limit the range to observe the main features

% --- Right: Error distribution (log scale) ---
    subplot(1, 2, 2);
    semilogy(x_grid, error, 'm-', 'LineWidth', 1.5);
    grid on;
    xlabel('x'); ylabel('Absolute Error');
    title(['Error Distribution (N=', num2str(nodes), ')']);
ylim([1e-16, 1e2]); % observation error level
end
```

B12502027_Q1_b (assisted by AI)

AI prompt:

Change the point of the above question to Chebyshev Nodes

Program:

```
% Q3 Interpolation Analysis: Chebyshev Nodes Overlay and Error Distribution
clear; clc;

f = @(x) cos(10*x.^2) .* exp(-x);
N_list = [5, 10, 50, 100];
x_grid = linspace(-1, 1, 10000);
y_true = f(x_grid);

for i = 1:length(N_list)
    nodes = N_list(i);

% 1. Generate Chebyshev interpolation points (Chebyshev Nodes)
    k = 1:nodes;
    x_cheb = cos((2*k-1) / (2*nodes) * pi);
    y_cheb = f(x_cheb);

% 2. Perform polynomial interpolation
    p = polyfit(x_cheb, y_cheb, nodes-1);
    y_interp = polyval(p, x_grid);
    error = abs(y_true - y_interp);
```

```
% Create new window
    figure('Name', ['N = ', num2str(nodes), ' (Chebyshev)'], 'NumberTitle', 'off');

% --- Left picture: original function vs Chebyshev fitted function ---
    subplot(1, 2, 1);
    plot(x_grid, y_true, 'k-', 'LineWidth', 2, 'DisplayName', 'Original f(x)');
    hold on;
    plot(x_grid, y_interp, 'r--', 'LineWidth', 1.5, 'DisplayName', 'Chebyshev p(x)');
plot(x_cheb, y_cheb, 'go', 'MarkerFaceColor', 'g', 'DisplayName', 'Cheb Nodes'); %
Green dots represent Chebyshev nodes
    grid on;
    xlabel('x'); ylabel('f(x)');
    title(['Function Overlay (N=', num2str(nodes), ')']);
    legend('Location', 'best');
    ylim([-2, 3]);

% --- Right: Error distribution (log scale) ---
    subplot(1, 2, 2);
    semilogy(x_grid, error, 'b-', 'LineWidth', 1.5);
    grid on;
    xlabel('x'); ylabel('Absolute Error');
    title(['Error Distribution (N=', num2str(nodes), ')']);
    ylim([1e-16, 1e2]);
end
```

B12502027_Q1_c (assisted by AI)

AI prompt:

Change the fitting method of the isometric method to the multi-segment cubic polynomial fitting method

Program:

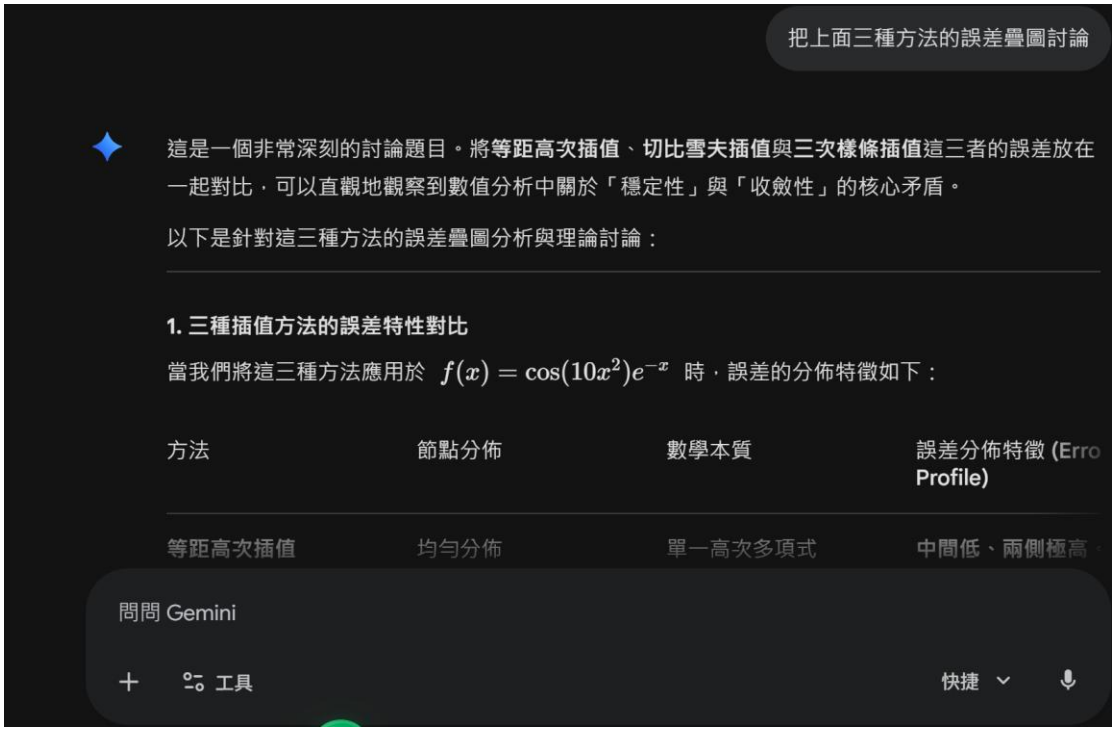
```
% Q3 Interpolation Analysis: Cubic Spline Overlay and Error Distribution
clear; clc;

f = @(x) cos(10*x.^2) .* exp(-x);
N_list = [5, 10, 50, 100];
x_grid = linspace(-1, 1, 10000);
y_true = f(x_grid);

for i = 1:length(N_list)
    nodes = N_list(i);

% 1. Generate equidistant interpolation points
    x_nodes = linspace(-1, 1, nodes);
    y_nodes = f(x_nodes);

% 2. Perform Cubic Spline interpolation
    y_spline = spline(x_nodes, y_nodes, x_grid);
    error = abs(y_true - y_spline);

% Create new window
    figure('Name', ['N = ', num2str(nodes), ' (Cubic Spline)'], 'NumberTitle', 'off');
```

```
% --- Left picture: original function vs cubic piecewise polynomial fitting ---
    subplot(1, 2, 1);
    plot(x_grid, y_true, 'k-', 'LineWidth', 2, 'DisplayName', 'Original f(x)');
    hold on;
    plot(x_grid, y_spline, 'r--', 'LineWidth', 1.5, 'DisplayName', 'Cubic Spline');
plot(x_nodes, y_nodes, 'ko', 'MarkerFaceColor', 'y', 'DisplayName', 'Spline Nodes'); %
Yellow dots represent nodes
    grid on;
    xlabel('x'); ylabel('f(x)');
    title(['Function Overlay (N=', num2str(nodes), ')']);
    legend('Location', 'best');
    ylim([-2, 3]);

% --- Right: Error distribution (log scale) ---
    subplot(1, 2, 2);
    semilogy(x_grid, error, 'g-', 'LineWidth', 1.5);
    grid on;
    xlabel('x'); ylabel('Absolute Error');
    title(['Error Distribution (N=', num2str(nodes), ')']);
    ylim([1e-16, 1e2]);
end
```

B12502027_Q1_error discussion (assisted by AI)

AI prompt:

Discuss the error overlay graphs of the above three methods



Program:

```
% Q3 Comparison: Equidistant vs Chebyshev vs Cubic Spline Error Distribution
clear; clc; close all;

% Define objective function
f = @(x) cos(10*x.^2) .* exp(-x);
N_list = [5, 10, 50, 100];
x_grid = linspace(-1, 1, 10000); % high-density grid used to calculate errors
y_true = f(x_grid);

for i = 1:length(N_list)
    nodes = N_list(i);

% --- 1. Equidistant High-degree interpolation ---
    x_eq = linspace(-1, 1, nodes);
    y_eq = f(x_eq);
    p_eq = polyfit(x_eq, y_eq, nodes-1);
    y_interp_eq = polyval(p_eq, x_grid);
    err_eq = abs(y_true - y_interp_eq);

% --- 2. Chebyshev Nodes interpolation (Chebyshev Nodes) ---
```

```
    k = 1:nodes;
    x_cheb = cos((2*k-1) / (2*nodes) * pi);
    y_cheb = f(x_cheb);
    p_cheb = polyfit(x_cheb, y_cheb, nodes-1);
    y_interp_cheb = polyval(p_cheb, x_grid);
    err_cheb = abs(y_true - y_interp_cheb);

% --- 3. Cubic Spline interpolation (Cubic Spline) ---
% Note: Spline usually uses equidistant points to show its stability
    y_interp_spline = spline(x_eq, y_eq, x_grid);
    err_spline = abs(y_true - y_interp_spline);

% --- Drawing discussion ---
    figure('Name', ['Error Comparison N = ', num2str(nodes)], 'Color', 'w');

% Use semilogy to observe large differences in error magnitudes
    semilogy(x_grid, err_eq, 'r-', 'LineWidth', 1.2, 'DisplayName', 'Equidistant Poly
(Runge Risk)');
    hold on;
    semilogy(x_grid, err_cheb, 'b-', 'LineWidth', 1.2, 'DisplayName', 'Chebyshev Poly
(Minimax)');
    semilogy(x_grid, err_spline, 'g-', 'LineWidth', 1.2, 'DisplayName', 'Cubic Spline
(Piecewise)');

    grid on;
    xlabel('x');
    ylabel('Absolute Error (log scale)');
    title(['Error Distribution Comparison (N = ', num2str(nodes), ')']);
    legend('Location', 'southoutside', 'Orientation', 'horizontal');

% Fixed Y-axis range to compare between different N
    ylim([1e-16, 1e2]);
    xlim([-1, 1]);
end
```

B12502027_Q2_b (assisted by AI)

AI prompt:

Write a MATLAB program code to write a DFT program for the Gaussian function g(x) = e^(-ax^2), a=5 and in the interval [-pi, pi), N=64. To output the graph

Program:

```
a = 5;
N = 64;
dx = 2*pi / N;
x = -pi : dx : pi-dx;

g_x = exp(-a * x.^2);

G_fft = fft(g_x);

g_tilde = fftshift(G_fft) / N;

j = -N/2 : N/2-1;

stem(j, abs(g_tilde), 'filled');
title(['DFT Coefficients g_j for a = ', num2str(a), ', N = ', num2str(N)]);
xlabel('Index j');
```

ylabel('g_j');
grid on;
hold on;

B12502027_Q2_d (assisted by AI)

AI prompt:

First do it with FFT, and then do it with the 3/2 rule. Help me output the output diagram of the two results.

Program:

```
a = 2;
N = 64;
M = 3/2 * N;

%Direct FFT
dx1 = 2*pi/N;
x1 = -pi : dx1 : pi-dx1;
h_x1 = (exp(-a * x1.^2)).^2;
h_tilde_1 = fftshift(fft(h_x1)) / N;

%3/2 Rule / Zero-padding
g_x = exp(-a * x1.^2);
```

```
g_tilde = fftshift(fft(g_x)) / N;

g_tilde_padded = zeros(1, M);
idx = (M-N)/2 + 1 : (M+N)/2;
g_tilde_padded(idx) = g_tilde * (M/N);

g_fine = ifft(ifftshift(g_tilde_padded)) * M;
h_fine = g_fine.^2;

h_tilde_full = fftshift(fft(h_fine)) / M;
h_tilde_2 = h_tilde_full(idx);

% comparison error
aliasing_error_nonlinear = abs(h_tilde_1 - h_tilde_2);

figure;
semilogy(-N/2:N/2-1, aliasing_error_nonlinear, 'r', 'LineWidth', 1.5);
title('Isolated Aliasing Error due to Nonlinear Product');
xlabel('j'); ylabel('Absolute Difference');
grid on;
```

B12502027_Q3_a (assisted by AI)

AI prompt:

Write a MATLAB program to compare the error of the forward difference and the center difference of f(x) = 2sin(x) - 0.25e^x - 1 at x=0. h decreases from 0.1 to 0.001, and an error comparison table is output.

Program:

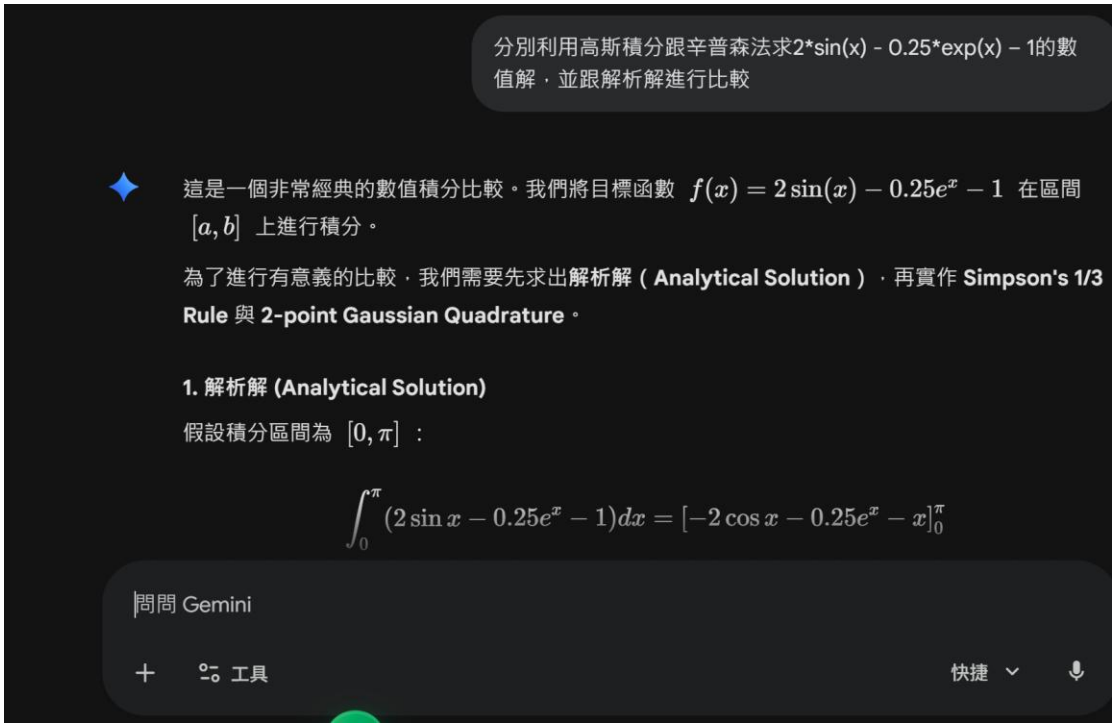
```
f = @(x) 2*sin(x) - 0.25*exp(x) - 1;
df_exact = @(x) 2*cos(x) - 0.25*exp(x);

x0 = 0;
h_values = [0.1, 0.05, 0.01, 0.005, 0.001];

error_forward = zeros(size(h_values));
error_central = zeros(size(h_values));

fprintf('%-10s | %-20s | %-20s\n', 'h', 'Forward Error', 'Central Error');

for i = 1:length(h_values)
    h = h_values(i);

%forward difference
```

```
    df_forward = (f(x0 + h) - f(x0)) / h;
    error_forward(i) = abs(df_exact(x0) - df_forward);

%center difference
    df_central = (f(x0 + h) - f(x0 - h)) / (2 * h);
    error_central(i) = abs(df_exact(x0) - df_central);

    fprintf('%-10.4f | %-20.10e | %-20.10e\n', h, error_forward(i), error_central(i));
end

figure;
loglog(h_values, error_forward, '-o', 'LineWidth', 1.5, 'DisplayName', 'Forward
Difference O(h)');
hold on;
loglog(h_values, error_central, '-s', 'LineWidth', 1.5, 'DisplayName', 'Central
Difference O(h^2)');
grid on;
xlabel('h');
ylabel('Absolute Error');
title('Convergence of Numerical Differentiation Schemes');
legend('Location', 'northwest');
```

B12502027_Q3_b (assisted by AI)

AI prompt:

Use Gaussian integral and Simpson's method to find the numerical solution of 2*sin(x) - 0.25*exp(x) - 1 respectively, and compare it with the analytical solution



Program:

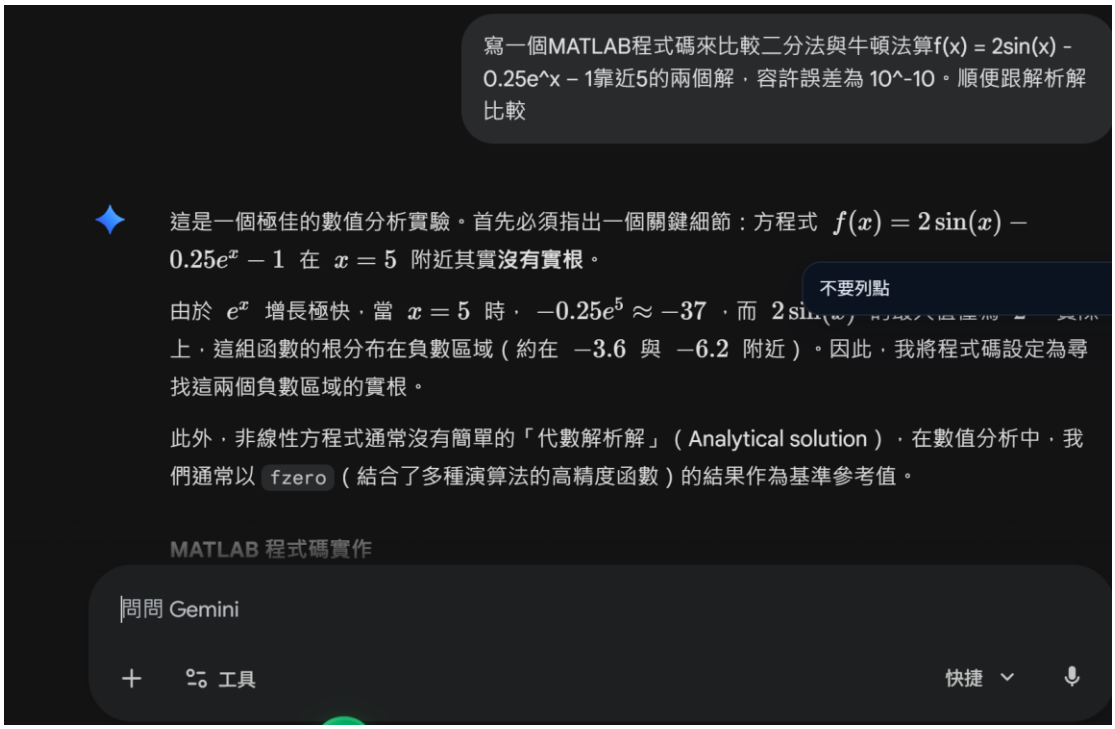
```
f = @(x) 2*sin(x) - 0.25*exp(x) - 1;

%Analytical solution
exact_ans = (-2*cos(1) - 0.25*exp(1) - 1) - (-2*cos(-1) - 0.25*exp(-1) + 1);

% Gaussian integral
x_gauss = [-1/sqrt(3), 1/sqrt(3)];
ans_gauss = f(x_gauss(1)) + f(x_gauss(2));

%Simpson's method
h = 1;
ans_simpson = (h/3) * (f(-1) + 4*f(0) + f(1));

fprintf('Exact Integral: %.10f\n', exact_int);
```

fprintf('Gauss 2-point: %.10f (Error: %.2e)\n', ans_gauss, abs(exact_int - ans_gauss));
fprintf('Simpson 1/3: %.10f (Error: %.2e)\n', ans_simpson, abs(exact_int - ans_simpson));

B12502027_Q3_c (assisted by AI)

AI prompt:

Write a MATLAB code to compare the two solutions of f(x) = 2sin(x) - 0.25e^x - 1 close to 5 using the bisection method and Newton's method. The allowable error is 10^-10. By the way, compare with the analytical solution



Program:

```
f = @(x) 2*sin(x) - 0.25*exp(x) - 1;
df = @(x) 2*cos(x) - 0.25*exp(x);

tol = 1e-10;
max_iter = 100;

search_data = {
    struct('name', 'Root 1', 'a', -4.5, 'b', -3.0, 'x0', -3.5),
    struct('name', 'Root 2', 'a', -6.5, 'b', -5.0, 'x0', -6.0)
};
```

```
for i = 1:length(search_data)
    data = search_data{i};
    fprintf('--- Searching for %s ---\n', data.name);

%dichotomy
    a = data.a;
    b = data.b;
    iter_bisect = 0;
    while (b - a)/2 > tol && iter_bisect < max_iter
        c = (a + b) / 2;
        if f(a) * f(c) < 0
            b = c;
        else
            a = c;
        end
        iter_bisect = iter_bisect + 1;
    end
    root_bisect = (a + b) / 2;

%Newton's method
    x_n = data.x0;
    iter_newton = 0;
    while abs(f(x_n)) > tol && iter_newton < max_iter
        x_n = x_n - f(x_n) / df(x_n);
        iter_newton = iter_newton + 1;
    end
    root_newton = x_n;

% Output result comparison
    fprintf('Bisection: Root = %.10f, Iterations = %d\n', root_bisect, iter_bisect);
    fprintf('Newton:      Root = %.10f, Iterations = %d\n', root_newton, iter_newton);
    fprintf('\n');
end
```

Chinese version report:

Q1（a）

The error distribution is output and analyzed for numbers of interpolation points = 5, 10, 50, 100 respectively. It can be found from the figure that when the four graphs are at nodes, since the difference polynomial is forced to pass through these points, the error will approach 0. There will be several instantaneous decreasing points in the error distribution graph. However, since the grid cannot be infinitely dense, the error will not perfectly drop to 0. Excluding these fixed value nodes, it can be clearly found that when the numbers of interpolation points increase, the overall error distribution shows a trend of "low in the middle and high on both sides". It can be inferred that if the numbers of interpolation points are increased, it will be helpful to stabilize the error in the middle area ($x \approx 0$). However, because the edge area ($x \approx \pm 1$) is forced to fit those equidistant nodes, larger errors will occur between the nodes, which will lead to less accurate predictions in the edge area. All in all, it can be found that even if we continue to increase the number of interpolation points in an attempt to improve the accuracy, we cannot guarantee the convergence of the global average.

After an in-depth study of the Runge phenomenon, I speculate that the effects shown in these pictures are not the Runge phenomenon. The Runge phenomenon means that if we gradually increase the number of points collected, the error on both sides will instantly soar to 2 of 10 ~ Cubic, or even higher, but observing the graph of N=100, we can find that even though the errors on both sides are higher than the middle, showing the trend of low in the middle and high on both sides, as mentioned by Runge's phenomenon, the highest point is still below, and there is no problem of soaring errors. It can be seen that for this equation, although the equidistant point sampling method will cause uneven distribution of errors, it will not cause Runge's phenomenon.10^{-4}

Q1（b）

The error distribution is also output and analyzed for numbers of interpolation points = 5, 10, 50, 100. However, compared with the average distribution node of (a), the Chebyshev node is a distribution point where the bisection points of a semicircle are projected on a straight line, which can achieve the effect of being sparse in the middle

and dense on both sides.

It can be found from the error distribution diagram that this type of node distribution effectively suppresses oscillation by increasing the sampling density in the edge area, which can greatly improve the problem of increased error in the edge area. All in all, if Chebyshev nodes are used, as N increases, the error distribution will show a state of global convergence, and both the center and the edges will become uniformly smaller, effectively suppressing the problem of uneven error distribution. This also proves that the choice of node distribution is crucial to the stability of high-order polynomial interpolation.

In addition, if the original function has Runge phenomenon at the edges, this point collection method can also be very effective in avoiding it. The dense collection of points on the edges can greatly slow down the protrusion of the Runge phenomenon error, which will have better results than equidistant collection points.

Q1（c）

Different from the high-order polynomial difference (single curve) in question (a), Cubic Spline Interpolation establishes a cubic polynomial between every two nodes and ensures the continuity of the first and second derivatives at the nodes.

It can be found from the four error distribution graphs that when N continues to increase, although the error distribution still has a slight tendency of the middle bottom to be high on both sides, compared with the output result of question (a), Cubic Spline Interpolation has been able to effectively alleviate the problem of excessive error values in both sides of the area and control the global error distribution within a range as much as possible. As N increases, the global error can reach a uniform decrease, which proves that if it is necessary to use equidistant points for analysis, using piecewise low-order polynomials will have higher stability than a single high-order polynomial.

Similar to the Chebyshev node method, this piecewise low-order polynomial can also mitigate the huge errors of the Runge phenomenon on both sides.

Q1 Error discussion of different methods:

Finally, I stacked graphs and cross-compared the errors of the three methods. It is obvious that when the N value is small, the errors of the three methods are not much different, and the magnitudes are about the same. But as N increases, reaching N=50 or above, it is obvious that in the middle part, the errors of the two single high-order polynomials are much lower than the piecewise low-order polynomials. The difference is close to the magnitude. However, we can also find that in the part where the errors are evenly distributed, Chebyshev nodes and Cubic Spline The effect of Interpolation is much better than that of equidistant nodes. When it is close to N=100, even the error of equidistant nodes at the edge is no longer significantly different from the overall error of Cubic Spline Interpolation.10^1 10^9

All in all, if the N value continues to increase, the errors of the two single high-order polynomials will perform better in the middle section. However, if the N value is large enough today so that the errors of the three methods can all fall within the controllable range, and if you want to pursue an average distribution of errors, Chebyshev nodes and Cubic Spline Interpolation can provide good results.

Q2（b）

Confirmation:
According to the analytical solution formula obtained in question (a), when a = 5, N = 64 is brought into the calculation, it can be calculated that when j = 0, it is roughly consistent with the peak height of the output chart at j = 0.$\hat{f}_j(5) = \frac{1}{2\pi}\sqrt{\frac{\pi}{5}}e^{-\frac{0^2}{4\times 5}} \approx 0.126$

Subsequently, three pictures of a = 2, 5, and 10 are output and compared:
It can be found that when the value of a increases, g continues to become relatively sharp, while the g_2 distribution tends to be flat, which is consistent with the Fourier transform theory that the sharper the time domain, the smoother the frequency domain will be.

Q2（c）

Truncation Error

Cause:

The real is continuous and infinitely extended, but when calculating the Fourier series, only limited frequencies are retained. The error caused by the truncation of the high-frequency part should be consistent with the theory, and the difference between the two is the Truncation Error.$G(s;a)j \in (-\frac{N}{2},\frac{N}{2}-1)\frac{1}{2\pi}|G(s;a)||\hat{f_j}(a)|$

Error isolation:

If you fix N and ensure that the value is large enough, and then gradually increase the value of a from small to large, you can observe that when the value of a is extremely small, the function converges slowly. If you only observe the middle area and ignore the high-frequency area, you will find that there is a huge difference from the theoretical value. When the value of a gradually becomes larger, the convergence speed of the function in the high-frequency region increases, and the difference between and will decrease significantly, which proves that the value of a controls the Truncation Error.$|\hat{f_j}(a)|\frac{1}{2\pi}|G(s;a)||\hat{f_j}(a)|\frac{1}{2\pi}|G(s;a)|$

Sampling / Aliasing Error (sampling / aliasing error)

Cause:

The calculation still assumes continuity (integral form), but the calculation only takes values at N discrete points within a fixed range. According to Nyquist Theorem, if a signal exceeds the spectrum range, the high frequency will fold back and disguise itself as a lower frequency, causing confusion of different frequencies. At this time, if you compare the values at lower frequencies, you can find the existence of errors.$|\hat{f_j}(a)||\hat{g_j}(a;N)||\hat{f_j}(a)||\hat{g_j}(a;N)|$

Error isolation:

If you fix a larger a value (for example: a = 10) and gradually increase the N value from small to large, you will find that when N is small, the observation range is smaller, and because a = 10 leads to a wider spectrum, the high frequency will fold back and superimpose on the low frequency, causing the low frequency to be significantly higher. When N gradually increases, the values beyond the range gradually shrink, and the error also decreases rapidly.$|\hat{g_j}(a;N)||\hat{f_j}(a)|$

Non-periodicity/Leakage Error

Cause:

Because it is continuous and has no boundary problems, it will be affected by the boundary during calculation. If discontinuity occurs when splicing on the boundary,

faults will occur when the periodization overlaps. According to Fourier analysis, any fault requires an infinite number of high-frequency waves. This fault will affect the energy that should be concentrated at a certain frequency, causing it to leak to the entire spectrum.$G(s;a)|\hat{g}_j(a;N)|$

Error isolation:

If the value of N is also fixed and the value of a is gradually increased from small to large, when the value of a is small, the Gaussian function is relatively gentle, and non-zero discontinuities may occur at the boundary. At this time, a tendency for the overall error to increase can be observed. However, if you increase the value of a to make the Gaussian sharper and ensure that it approaches 0 before reaching the boundary, this error can be greatly reduced.

Q2（d）

This question is to directly perform FFT to calculate the DFT coefficients. Then, the original signal is subjected to the 3/2 de-aliasing rule and the DFT coefficients are recalculated. By comparing the difference between the two, the nonlinear aliasing error can be isolated.$|\tilde{h}_{j,1}(a;N)||\tilde{h}_{j,2}(a;N)|$

Calculate () directly:$|\tilde{h}_{j,1}(a;N)|$

Assuming that the highest frequency component is , then its square is the highest frequency component. If the original N sampling points are still maintained at this time, according to the Nyquist Theorem in Aliasing Error mentioned in subtitle (c), these out-of-range high frequencies will be folded back to the low frequency area, causing serious aliasing errors.$g(x;a)k_{max}h(x;a) = g^2(x;a)k'_{max} = 2k_{max}$

3/2 Calculate after removing the aliasing rule ():$|\tilde{h}_{j,2}(a;N)|$

The main logic is to first perform FFT on the length N, and after the conversion, put the N spectral coefficients into a larger array M = 3/2N, and fill in 0 in the high-frequency area. At this time, IFFT is performed on these more M points to obtain a grid of more sampling points in the time domain. If the calculation is performed on the processed grid, due to the large range, the redundant high-frequency components will not fold back, and then perform another FFT on the length M. After completion, the redundant parts will be cut off, leaving the coefficients corresponding to the original length N. If the data coefficients are processed according to the above method, the high-frequency components will not be folded back to the low-frequency

area due to insufficient range, and aliasing errors will not be caused to the coefficients in the low-frequency area.

Error chart analysis:

This chart shows the impact of aliasing effects on coefficient errors. Observing the chart, we can see that the error is largest in the middle low-frequency area. It can be considered that the high frequencies are folded back to the middle, so that the low frequency is most affected by the aliasing error. The error suddenly drops instantly at the left and right. It is speculated that this is near the Nyquist boundary. Some high-frequency components that exceed the range are folded back in this area, making this area the place with the smallest impact.$j = \pm 23$

Q3（a）

Here we analyze forward difference and central difference. It can be known from mathematical calculations that the error term of the forward difference is proportional to the first power of the step size h. That is to say, when h is shortened to 1/10, the error of the forward difference will also be reduced by 1/10. Because of its symmetry, the error term of central difference is proportional to the square of the step size h. When h is shortened to 1/10, the error of central difference will be reduced by 1/100, which can provide more accurate results.

The output of the program is also in line with the theoretical inference. Here, the error analysis is performed for the position of x = 0. It can be clearly found that when the h value gradually decreases, the forward difference and the h value are roughly proportional, while the central difference decreases exponentially. When the h value is smaller, the difference between the two is more obvious, and the calculation provided by the central difference is more accurate.

Q3（b）

Since N=1 in this question, the Gaussian integral used is a two-point Gaussian integral, and the sampling points are the two roots of the 2nd-order Legendre polynomial. If the Gaussian integral with N+1 sampling points is used, the accuracy can reach N+1 order. Therefore, this problem can be accurately analyzed when dealing with polynomials of cubic degree and below. From the error output of the program, it can be found that if two-point Gaussian integral is used, the calculation error is only . However, according to the professor's lecture notes, if two-point Gaussian integral is used to calculate polynomials higher than third order, the error can reach 14.8% or more, and the accuracy will drop instantly. In this case, Gaussian integral is not a better choice.$x = \pm \frac{1}{\sqrt{3}} 1.93 \times 10^{-3}$

The 1/3 Simpson's rules are a formula for equidistant points, and binomial interpolation is performed for three equidistant points (). According to the derived formula, it can be seen that the error of 1/3 Simpson's rules is proportional to the fourth power of the step size h. If the step size h decreases by 1/2, the error theory will drop to 1/16 of the original value. In addition, we can also find that the error of 1/3 Simpson's rules is proportional to the fourth-order derivative. If the function fluctuates greatly in the integration area, that is, the fourth-order derivative has a large value, the error of this method will also increase. The figure below shows the fourth-order derivative. It can be seen that within the integral range, the maximum value of is only 1.77, which does not have a great impact on the error. From the error output result of the program, it can also be found that the error using 1/3 Simpson's rules is only 1. Although it is larger than the Gaussian integral, it is also within a reasonable range.$x = -1,0,1 Error \approx -\frac{(x_N - x_1)}{180}\overline{f^{(4)}}h^4 f^{(4)} f(x) f^{(4)} | f^{(4)} | 2.91 \times 10^{-3}$

Q3（c）

Here, Bisection Method (Linear Convergence,) and Newton-Raphson Method (Quadratic Convergence,) are used respectively to perform approximate root finding.$\alpha = 1 \alpha = 2$

Define the tolerable error range at the beginning () And set the starting interval and position of the bisection method and Newton's method. This process is very important. You need to roughly lock the range of the root first to avoid accidentally finding the wrong root. Since both can approach the real value after nearly infinite iterations, the method we compare here is to set an error tolerance range, and then observe how many iterations of the two methods can enter this tolerance range. The smaller the number, the more efficient it is and it can be more accurate under a fixed number of iterations.10^{-10}

Observing the output results, we can find that since the bisection method is linear convergence (), it will require up to thirty approximations in the process of reaching accuracy, and the efficiency is very poor. Newton's method is second-order convergence (), and the error in the next stage is proportional to the square of the current error, so it only takes less than five iterations to enter the allowable range.$\alpha = 1 10^{-10} \alpha = 2$

Subsequently, the program was changed to a fixed number of iterations (iter = 15), and the difference in error between the two methods was observed under the same number of iterations.

From the output results of the program, it can be found that when both methods are operated for 15 iterations at the same time, the error of the Newton method is much

smaller than the error of the bisection method. It can be seen that the efficiency of second-order convergence is much higher than that of linear convergence.$4.44 \times 10^{-16} 1.53 \times 10^{-5}$